



Why Phase 2 Had to Become Agentic

Phase 1 was a capable tool but lacked core agent properties. Phase 2 transforms SmartDoc into a goal-driven, learning agent that perceives, decides, acts, and learns—with human oversight and auditability.

Agent Properties — 4-Quadrant Overview



PERCEPTION

Sense the environment: brightness, contrast, sharpness, noise, skew, document type, dark/ruled-line detection.



DECISION-MAKING

Selects best action: chooses basic / enhanced / aggressive pipelines based on quality score and memory signals.



ACTION

Executes the plan: preprocessing, OCR, structure detection, and .docx generation with formatting preserved.



LEARNING

Improves from experience: feedback updates strategy weights in persistent JSON memory for future decisions.

Gap Analysis — Phase 1 → Phase 2

Phase 1 addressed a small subset of agent needs. Phase 2 closes nearly all operational gaps to become agentic.

Requirement	Phase 1	Phase 2
Image perception	✗ None	✓ 6 quality metrics
Strategy selection	✗ Fixed	✓ basic/enhanced/aggressive
Decision explanation	✗ Hidden	✓ Per-line reasoning
Short-term memory	✗ None	✓ session_state
Long-term memory	✗ None	✓ JSON persistence
Learning loop	✗ None	✓ Feedback → memory update
Human override	✗ None	✓ At every step
Audit trail	✗ None	✓ Downloadable JSON

📌 Big takeaway: Phase 1 covered 1/10 agentic requirements. Phase 2 closes 9/10.

Our Agentic Vision: Goal-Based + Learning

Move from a static tool to a collaborative agent that optimizes for a single, measurable goal while improving with feedback.

Transformation

PHASE 1 (TOOL): User → System → User — one-way, blind, static. PHASE 2 (AGENT): Agent $\rightleftarrows$ Human — collaborative, explainable, adaptive.

Key Shifts

- Reactive → Proactive: warns and suggests strategies
- Static → Adaptive: learns across sessions
- Opaque → Transparent: per-line explanations

Agent Type Decision

Simple reflex and model-based agents lack learning or memory. Utility agents are overcomplex. We chose Goal-Based + Learning for targeted, incremental improvement.

Operating Goal

Produce the best-quality formatted Word document with minimum user effort — and improve every session.

 Why semi-autonomous? Handwritten OCR ≈ 60–70% — human review remains essential for responsible quality.

SmartDoc OCR — 6-Agent Architecture

End-to-end agent pipeline: perception → preprocess → OCR → formatting decision → document generation → feedback + memory.
Human can override at every step.



Human-in-the-Loop

Sidebar override available at every stage — set engine, adjust thresholds, replay logs, reset memory, and download audit JSON.

Observe → Interpret → Decide → Act → Learn — Real Run Example

1. OBSERVE

Image metrics: brightness 142/255 · contrast 87 · sharpness 1240 · noise 12% · skew 1.2° · handwritten chemistry notes.

2. INTERPRET

Quality score = 58/100 (moderate).
Challenge: low brightness + handwriting → high difficulty.

3. DECIDE

Quality & memory → choose **aggressive** preprocessing (aggressive_wins=5 increases confidence). No user override.

4. ACT

Run aggressive pipeline + Tesseract → 213 words at 64% average confidence; detected 8 headings, 12 bullets, 3 centered lines → produced 45 KB .docx.

5. LEARN

User rated 4/5 → aggressive_wins += 4. Agent updates memory and will prefer aggressive for similar inputs next session.

✅ Result: clean formatted Word document in ~7 seconds end-to-end. Agent is measurably smarter for the next run.

How SmartDoc OCR Thinks — 3-Tier Intelligence

Tier 1 — RULE-BASED

- Deterministic checks: startswith('#') → Heading; ALL CAPS heuristics; bullet markers; center_x alignment; avg char height → Bold.

Tier 2 — HEURISTIC / STATISTICAL

- Quality score = weighted blend (brightness, contrast, sharpness, noise).
- Strategy gating: $\geq 65 \rightarrow$ enhanced; $< 65 \rightarrow$ aggressive; dark-bg $\rightarrow$ invert first.
- Multi-PSM selection: choose PSM by $\text{word_count} \times 0.5 + \text{avg_conf} \times 0.5$.

Tier 3 — LEARNING

User rating $\times$ strategy stored in smartdoc_memory.json; strategy_wins counters adapt recommendations; memory persists across sessions.

Future enhancement: optional LLM (Gemini) layer for semantic structure extraction.



Memory — How SmartDoc Remembers and Learns

Short-Term Memory

Storage: Streamlit session_state. Contains pipeline metrics, audit log, last strategy, current doc buffer. Lifetime: current browser session. Purpose: pass context between agent steps during a conversion.

Long-Term Memory

Storage: smartdoc_memory.json (/tmp or local). Contains total sessions, strategy_wins (basic:2, enhanced:8, aggressive:5), feedback history with timestamps. Persists across sessions to enable cross-session learning.

Memory in Action — 3-session example

1. Session 1: Dark image → enhanced → OCR poor → user rates 2/5 → enhanced_wins +=2
2. Session 2: Similar image → memory favors aggressive → better OCR → user rates 4/5 → aggressive_wins +=4
3. Session 3: Agent auto-selects aggressive → faster, higher quality result.

❏ After 3 sessions, agent selection becomes measurably smarter than initial baseline.

Autonomy Level: Semi-Autonomous (By Design)

Phase 2 targets a 50/50 split: the agent suggests and optimizes; humans verify and decide. This balance preserves quality while enabling learning.

1

1. Before perception

Human uploads image, optionally pick engine or title.

2

2. After perception

Human reviews quality metrics and can override strategy.

3

3. After OCR

Review confidence and abort or continue.

4

4. After formatting

Inspect per-line reasoning and edit decisions.

5

5. After download

User rates output → memory updates.

"The agent suggests. The human decides. The agent learns."

Phase 1 vs Phase 2 — The Transformation Achieved

Feature	Phase 1 → Agentic Phase 2
Control	User-driven (manual) → System-driven + human override
Intelligence	Static rules → Adaptive (rules + heuristics + memory)
Behavior	Reactive → Proactive (analyze, recommend, warn)
Preprocessing	One fixed pipeline → Dynamic basic/enhanced/aggressive
OCR engines	Tesseract only → Tesseract + EasyOCR (handwriting)
Memory	None → Short-term + long-term JSON
Learning	None → Feedback updates strategy weights
Transparency	Hidden → Per-line reasoning + audit log
Architecture	Monolithic → 6 distinct agent classes
Audit trail	None → Downloadable JSON

Results We Achieved

- Both apps deployed live on Streamlit Cloud
- OCR accuracy improved 40% → 70%+ with adaptive preprocessing
- Handwriting support added via EasyOCR integration
- Dark-background and colored notebook paper handled (auto-invert, HSV ruled-line removal)
- Full agent transparency with downloadable logs
- Verified end-to-end in dry runs and production
- Public repo: github.com/mmoezjamal-ai/ppit_project

✔ Phase 1 was a powerful tool. Phase 2 is an intelligent, responsible, learning agent — aligned with 2026 industry expectations.

Thank you. Questions?